تقرير التدقيق المستقل — مستودع sedeek7712/mhdl

نظام عمرو النماري للصرافة · الفرع: main · Commit: `0091a23f55970a4237283b82c34a8c3837cf8a8f`
تدقيق للقراءة فقط: لم يُعدَّل أي ملف في المستودع الأصلي.

1. الفرع وCommit

- لا يوجد إلا `main`، وآخر Commit هو `0091a23`. الفرع `fix/financial-security-and-audit` مدمج ولا أثر له منفصلاً.
- التاريخ خطي من 5 Commits.

2. ملخص البنية

- خادم Express (`server.ts`) وواجهة React/Vite.
- الخادم يحمل سجل الحوالات والصرف وواتساب فقط، ويحفظها في ملفات JSON داخل `data/`.
- القيود والصناديق والعملاء موجودة في المتصفح (`App.tsx` + `localStorage`).
- لا يوجد Electron إطلاقاً.

3. المشكلات المكتشفة

حرجة

- **C1. تزوير توكن المالك.** `npm start` يشغّل `node server.ts` دون `NODE_ENV=production`، فيُستعمل مفتاح JWT الافتراضي المكتوب في الكود (`authService.ts:30-40`). جرّبت: توكن owner وقّعته بهذا المفتاح قبلته `verifyToken`. الأثر: كل الصلاحيات لأي شخص يصل للمنفذ.
- **C2. الكاشير يختلق حوالة ويصرفها.** مسار `/api/remittances/register-master` متاح للكاشير ويقبل `amount` و`status:'available'` من الطلب، ثم يصرفها `payout`. جرّبت: حوالة وهمية بمليون سُجلت وصُرفت بنجاح. المطابقة `reconcilePayouts` لن تكشفها لأنها تقارن حقولاً كتبتها الدالة نفسها.
- **C3. الصرف بلا قيد حقيقي على الخادم، والمحاسبة منقسمة.** الخادم يولّد مرجعاً وهمياً `سند-رقم عشوائي-JV` ولا يكتب قيداً. القيد الحقيقي يُكتب في المتصفح برقم مختلف بعد نجاح الخادم (`App.tsx:~616-735`). عند انقطاع الاتصال أو إغلاق التبويب بعد نجاح الخادم تصبح الحوالة مصروفة بلا قيد ولا خصم من الصندوق، والمحاولة الثانية تُرفض بـ"مصروفة مسبقاً" لأن مفتاح الـ idempotency يحوي `Date.now()`.

عالية

- **H1.** `managerOverride` و`cashierDrawerBalance` يأتيان من العميل؛ أي كاشير يتجاوز فحص السيولة ويُسجَّل باسمه "معتمداً".
- **H2.** مفتاح Idempotency غير مربوط بالحوالة: جرّبت نفس المفتاح لحوالتين، فأرجع الخادم نجاحاً للثانية وسجل الأولى، وبقيت الثانية `available`.
- **H3.** رمز PIN قابل للكسر: التجزئة والملح في المصدر (`SYSTEM_USERS`)، ومساحة 4 أرقام تُكسر دون اتصال في نحو 3 دقائق لكل مستخدم (16ms للتخمين). رمز المالك مكتوب أيضاً في ملف الاختبارات (تحققت من تطابقه).
- **H4.** التخزين يضيّع البيانات: `writeFileSync` مباشر دون ملف مؤقت أو rename، فالإغلاق أثناء الحفظ يترك JSON مقطوعاً؛ وعند القراءة يبتلع `catch` الخطأ ويستعمل الافتراضيات ثم يكتب فوق الملف، فيرجع `nextSequence` إلى 931107 وتتكرر الأرقام. والصرف يكتب ثلاثة ملفات متتالية بلا Transaction.
- **H5.** `/api/auth/users` بلا مصادقة ويكشف الأسماء والأدوار، والخادم يستمع على `0.0.0.0:3000` مخالفاً قرار "جهاز واحد محلي".
- **H6.** فشل Meta الحقيقي يسقط صامتاً إلى وضع Sandbox ويسجّل `SENT` ثم `DELIVERED` دون إشعار بالفشل.

متوسطة

- M1. المبالغ لا تُتحقق: `amount!` يمرّر السالب (الفحص على التعبير فقط، لم يُختبر عبر HTTP)، ولا تحقق للعملة ولا لـ `feeAmount`.
- M2. لا تحقق من تطابق المستلم مع الحوالة المسجلة ولا من اتجاه الحوالة.
- M3. التوكن يُقرأ من `localStorage` أيضاً (`apiClient.ts:17`).
- M4. "التخزين الآمن" في المتصفح هو `btoa` وليس تشفيراً.
- M5. سجل التدقيق ملف JSON قابل للتعديل بلا سلسلة تجزئة، وسجل الأمان في الذاكرة يضيع عند إعادة التشغيل.
- M6. توكن webhook الافتراضي مكتوب في الكود، والتحقق من توقيع Webhook يُتخطى خارج الإنتاج.
- M7. `register-number` يحجز أرقاماً دون ربطها بحوالة، وينهار `.trim()` إن غاب الرقم.

منخفضة

- L1. أوقات `paidAt` بتوقيت UTC دون تنبيه (اليمن UTC+3).
- L2. بيانات شبه حقيقية وصور سندات اختبار في `data/` وغير مستثناة في `.gitignore`.
- L3. مجلدا `.aistudio` و`dev-dist/` ضمن Git.

4. الجاهزية

- **Windows EXE: غير جاهز.** لا Electron، ولا مسار بيانات خارج مجلد التثبيت (`DATA_DIR` نسبي للمصدر فيُمسح مع التحديث)، ولا نسخ احتياطي واستعادة على الخادم. الطباعة تعتمد على المتصفح.
- **يعمل دون إنترنت:** الصرف والقيود والسندات. أما واتساب فيحتاج إنترنت.
- **SQLite:** البنية مناسبة جزئياً لأن الخادم يحمل منطق الحوالات؛ لكن يجب نقل الحسابات والعملاء من المتصفح أولاً ليوجد مصدر حقيقة واحد.

5. مقترح SQLite (للتقييم فقط)

- **الجداول:** `users`، `remittances` (UNIQUE على رقم الحوالة)، `payouts` (UNIQUE على `remittance_id`)، `idempotency_keys` (UNIQUE على المفتاح+المستخدم مع بصمة الطلب)، `journal_entries`، `journal_lines`، `accounts`، `cash_movements`، `audit_log` (إضافة فقط بتجزئة متسلسلة)، `settings`.
- **تحتاج Transaction واحدة:** الصرف (الحالة + السجل + القيد + حركة الصندوق + التدقيق)، وتوليد الرقم التسلسلي، وتسجيل الحوالة.
- **الترحيل:** نسخة كاملة من `data/` أولاً، ثم استيراد داخل Transaction واحدة، ثم مطابقة (الأعداد ومجاميع المبالغ لكل عملة وتجزئة كل سجل)؛ عند أي فشل يُلغى الاستيراد وتبقى JSON كما هي.

6. الملفات والاختبارات

- **قُرئت بالكامل:** `server.ts`، `authService.ts`، `remittanceRegistry.ts`، `package.json`، `.gitignore`، `.env.example`.
- **قُرئت أجزاء:** `metaWhatsAppService.ts`، `dispatchRegistry.ts`، `offlineStorage.ts`، `apiClient.ts`، `accountingEngine.ts`، `App.tsx` (مسار الصرف)، وملفات `data/`.
- **لم تُفحص:** مكونات الواجهة، `voucherEngine`، `notificationEngine`، `whatsappService`، `printAuditService`.
- **الاختبارات:** شُغّل `tests/enterprise_audit_verification.ts` على نسخة معزولة فنجح 16/16، لكنه لا يغطي C1 وC2 وC3 وH1 وH2، ونجاحه لا يعني سلامة المحاسبة أو الأمان.
- **تجارب إضافية** (على الطبقة الخدمية مباشرة لا عبر HTTP): تزوير التوكن، اختلاق الحوالة وصرفها، تكرار مفتاح الـ idempotency.

- **حدود:** فشل `npm install` الكامل بتعارض اعتمادات (ثُبّت `tsx` وexpress فقط). لم يُشغَّل `tsc` ولا `vite build`، ولا Electron لأنه غير موجود.

7. خطة الإصلاح المقترحة

1. إلغاء المفتاح الافتراضي وإلزام `JWT_SECRET`، وربط الخادم بـ `127.0.0.1`، وإغلاق `/api/auth/users`.
2. إزالة تحكم العميل في الحالة والمبلغ و`managerOverride` و`cashierDrawerBalance`، وحصر تسجيل الحوالة بدور مخول.
3. نقل القيد والصندوق إلى الخادم داخل عملية الصرف نفسها، وربط الـ idempotency بالحوالة والمستخدم وبصمة الطلب.
4. كتابة ذرية للملفات (مؤقت ثم rename) ورفض بدء التشغيل عند تلف JSON بدل الرجوع للافتراضيات.
5. كلمات مرور أطول وتخزين المستخدمين خارج الكود.
6. إزالة الرجوع الصامت إلى Sandbox في واتساب.
7. بعدها: SQLite، ثم حزمة Electron مع البيانات في `%APPDATA%`.

لا يُنفَّذ أي إصلاح قبل موافقة صريحة.